

# BoloStock — Voice-Based Inventory Management for Small Businesses

## Milestone 1.1 · Requirements Gathering & Analysis

**Deliverable:** Requirements Document

**Date:** 19 September 2026

**Prepared by:** \_\_\_\_\_

---

### Project Context

**BoloStock** is a voice-first inventory system for small Indian retail shops.

Every voice inventory product on the market works as a **command interface**: you press a button and issue a structured instruction — *"add twenty kilo rice."* That is still data entry, performed with your mouth instead of your thumbs. BoloStock inverts this. The phone sits on the shop counter and **listens to the shop**, extracting stock movements from the ordinary conversation between owner and customer — sentences that were never addressed to the app at all. A kirana shop already says every transaction out loud, usually twice: once when the customer asks and once when the owner confirms. That speech is a complete, free transaction log that currently evaporates into the air. **BoloStock captures it.**

Because passive capture is less accurate than dictated commands, nothing it extracts is written to stock automatically. Every extraction becomes a **proposal** the owner approves in bulk, and a conventional **Command Mode** mic button remains available as the reliable path. The novel capability is genuinely novel; the failure mode is an ordinary voice inventory app.

### Milestone 1 deliverables

- **1.1** — Requirements Gathering & Analysis (*this document*)
- **1.2** — Functional Requirements
- **1.3** — Technical Design & Architecture
- **1.4** — UI/UX Design & Prototype

---

## Requirements Gathering & Analysis

### 1. Problem Context

India has roughly 13 million kirana and small trade shops. The overwhelming majority track stock in one of three ways: a paper notebook (*bahi khata*), the owner's memory, or ad-hoc WhatsApp messages to a supplier. All three fail identically — **the record is not queryable.** The owner cannot answer "how much rice is left?" without walking to the godown, and cannot answer "what do I need to order?" at all until something has already run out.

Existing digital inventory software does not solve this, because it moves the work to a keyboard. It assumes the operator can type, can type in English, can read terms like *SKU*, *variant*, and *reorder point*, and has patience for a multi-field form per entry. An owner logging 40 stock movements a day will not do this.

**Voice-command inventory apps are a partial fix, and they are already widely available.**

They remove the keyboard but keep the *interruption*: the owner must stop serving, open the app, tap a button, and speak a well-formed sentence. In a shop with a queue at the counter, that interruption is the reason the feature goes unused after week one.

**The observation that drives this project:** the transaction has already been spoken aloud, in natural language, by two people, before any app was opened. The data-entry step is redundant. The system should listen instead of asking.

### 2. Target Users

| Persona                                   | Profile                                                                                     | Primary need                                      |
|-------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------|
| Ramesh, 48 — kirana owner, Tier-2 city    | 8th standard, fluent Hindi, minimal English, Android phone, uses WhatsApp voice notes daily | Stock to be recorded <i>without stopping work</i> |
| Lakshmi, 35 — provisions store, Telangana | Speaks Telugu with English product names mixed in, reads numerals but not English words     | Know what is running out before customers ask     |
| Arjun, 22 — owner's son, does the         | Comfortable with apps                                                                       | A reliable reorder list, not a guess              |

|          |  |  |
|----------|--|--|
| ordering |  |  |
|----------|--|--|

**Common denominator:** all three are fluent *speakers*, reluctant *typists*, and — critically

— already narrate every sale out loud as part of doing business.

### 3. Evidence Basis

Requirements derive from the GigPoint problem statement and three observable behaviours:

1. **The transaction is spoken before it is recorded.** A counter sale is a verbal exchange:

the customer names the item and quantity, the owner confirms. This happens 100% of the time, with no prompting and no app.

2. **Speech is code-switched, not monolingual.** A real utterance is "*do carton Maggi dena*"

— a Hindi numeral, an English trade unit, a brand name, a Hindi verb. Any system demanding one language per sentence will be abandoned.

3. **Trade units are the mental model.** Stock is thought of in *bori*, *peti*, *dozen*, *tin*,

*quintal*. Forcing entry in kilograms makes the owner do arithmetic — exactly the friction being removed.

### 4. Current-State Pain → Requirement Mapping

| Pain observed                       | Consequence                             | Requirement generated                                           |
|-------------------------------------|-----------------------------------------|-----------------------------------------------------------------|
| Entry requires typing               | Stock logged late or never              | R1 — voice as primary input                                     |
| Even voice entry interrupts serving | Feature abandoned after initial novelty | <b>R2 — passive capture; zero interaction in the happy path</b> |
| Software is English-only            | Owner needs a helper to operate it      | R3 — regional + mixed-language input                            |
| Units forced to kg/pieces           | Mental arithmetic on every entry        | R4 — accept trade units, convert internally                     |
| No visibility of totals             | Shortages found at point of sale        | R5 — at-a-glance dashboard                                      |
| No reorder signal                   | Over-ordering and stockouts together    | R6 — threshold alerts + reorder list                            |
| Speech recognition is imperfect     | A wrong entry silently corrupts stock   | <b>R7 — nothing auto-commits; propose, then bulk-approve</b>    |
| Notebook can be lost                | Total history loss                      | R8 — durable server-side persistence + history                  |

### 5. Scope

**In scope (v1, delivered today)**

- **Counter Mode** — continuous passive listening that proposes stock movements from

overheard conversation

- **Command Mode** — a mic button for deliberate, single, well-formed entries
- Hindi, Telugu and Indian-English input, freely mixed within one sentence
- Trade-unit entry with per-item conversion to a base unit
- Pending-proposal tray with bulk accept/reject
- Stock dashboard, low-stock alerts, reorder suggestions in purchase units
- Spoken and written confirmation in the owner's language
- Full transaction history retaining the original transcript per entry
- Server-side persistence

### Out of scope (explicitly deferred)

Billing and GST invoicing · customer credit (*udhaar*) ledger · multi-shop and staff roles ·

barcode or image entry · offline-first sync · demand forecasting beyond a threshold rule ·

supplier ordering integration · speaker diarisation (telling owner and customer apart)

## 6. Constraints

| Constraint                        | Consequence for design                                                |
|-----------------------------------|-----------------------------------------------------------------------|
| Zero budget                       | No paid speech APIs; browser-native recognition only                  |
| No model training or fine-tuning  | Recognition off-the-shelf; understanding rule-based and fuzzy-matched |
| Single developer, one working day | Narrow and vertically complete, not broad                             |
| Low-digital-literacy users        | No software jargon in any visible string; 48 px minimum touch targets |
| Low-end Android, patchy network   | Mobile-first, small payloads, graceful degradation to typed input     |
| Counter device is on a charger    | Continuous listening is acceptable; a pocket app it is not            |

## 7. Assumptions

4. The shop has a smartphone with Chrome (Android) or Safari (iOS) and intermittent internet.
5. A shop stocks 20–100 distinct items — small enough for exhaustive matching against a

per-shop catalogue rather than open vocabulary.

6. Counter conversations are audible to a phone placed on the counter.
7. Pack sizes (*1 bori rice = 50 kg*) are configured once at setup and change rarely.
8. The owner will review a batch of proposals once or twice a day, rather than confirm each

entry in the moment.

## 8. Risks

| Risk                               | Likelihood | Impact | Mitigation                                                            |
|------------------------------------|------------|--------|-----------------------------------------------------------------------|
| Passive extraction accuracy is low | High       | High   | Nothing auto-commits. Proposals go to a review tray; a wrong proposal |

|                                                            |        |        |                                                                                                                                          |
|------------------------------------------------------------|--------|--------|------------------------------------------------------------------------------------------------------------------------------------------|
|                                                            |        |        | costs one tap to dismiss, not a corrupted ledger                                                                                         |
| <b>Conversation contains no usable transaction</b>         | High   | Low    | Low-confidence output is discarded silently rather than shown                                                                            |
| <b>Continuous recognition stops on silence or tab blur</b> | High   | Medium | Restart-on-end loop; a visible "listening" indicator so the owner can see it died                                                        |
| <b>Browser lacks Web Speech API</b>                        | Medium | High   | Editable transcript field feeds the identical parse endpoint                                                                             |
| <b>Privacy objection to listening at the counter</b>       | Medium | High   | Recognition runs in the browser; <b>no audio is recorded, stored or uploaded</b> — only extracted ( <i>item, qty, direction</i> ) values |
| <b>Battery drain / constant network</b>                    | High   | Medium | Positioned as a counter-side device on a charger                                                                                         |
| <b>Ambiguous unit ("1 bag")</b>                            | Medium | Medium | Pack size stored per item; unknown unit triggers a clarification chip                                                                    |

## 9. Success Criteria

On a live phone against the deployed URL:

9. A performed counter conversation produces at least one correct stock proposal without any button being pressed.
10. A Telugu–English mixed sentence is parsed correctly for item, quantity and unit.
11. A trade unit ("*do bori*") is stored correctly converted to the base unit.
12. Every proposal can be accepted or corrected without opening the keyboard.
13. "What is running low?" returns a correct, spoken answer.
14. Stock totals survive a page reload and a server restart.

## 10. Non-Goals

This is **not** a general-purpose voice assistant. It deliberately understands a closed

domain — the items in one shop's catalogue plus a fixed set of trade units and actions. That

narrowness is precisely why it can be accurate without any trained model, and it is a design

decision rather than a limitation of the timeline.